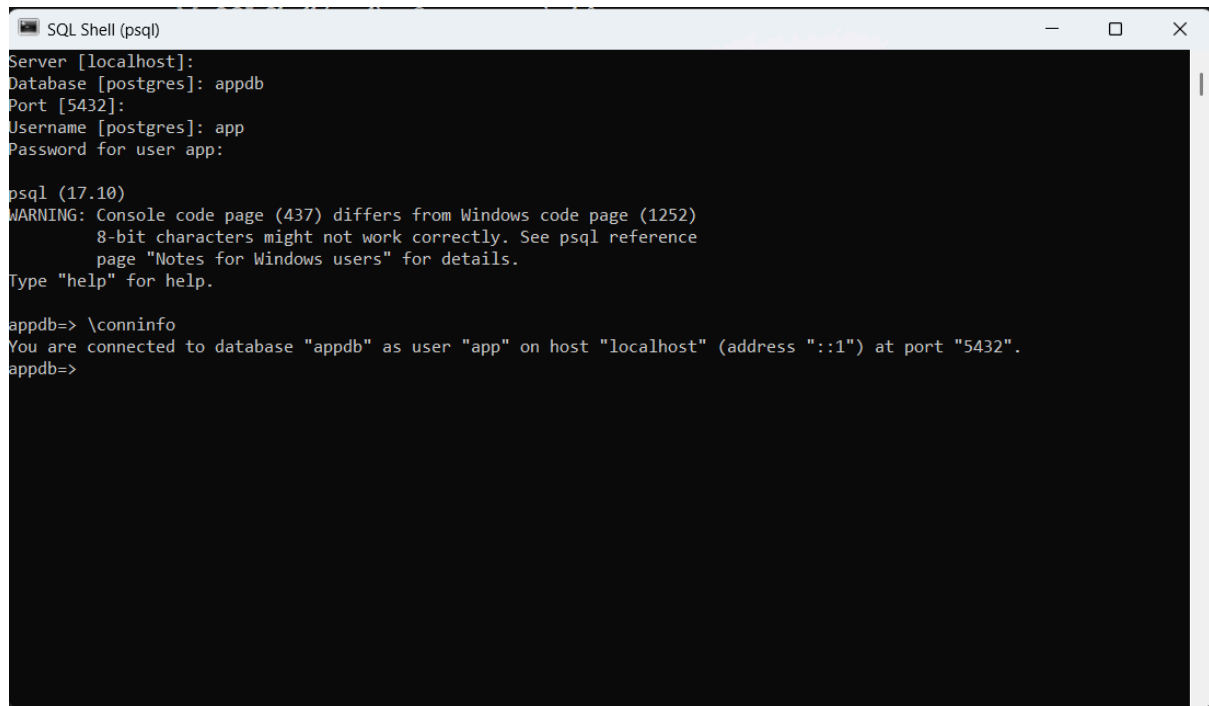


Name: Dương Huy Phúc

link github: <https://github.com/phuc0401/database-web>

Check point 0:



```
SQL Shell (psql)
Server [localhost]:
Database [postgres]: appdb
Port [5432]:
Username [postgres]: app
Password for user app:

psql (17.10)
WARNING: Console code page (437) differs from Windows code page (1252)
        8-bit characters might not work correctly. See psql reference
        page "Notes for Windows users" for details.
Type "help" for help.

appdb=> \conninfo
You are connected to database "appdb" as user "app" on host "localhost" (address "::1") at port "5432".
appdb=>
```

Part 1 — SQL Warm-up: Tables, Keys, Relationships (20')

Task 1. Write SQL queries for: 1. All heroes aged 18 or older, oldest first.

```
select name, age from hero
where age > 18
order by age desc;
```

2. Each hero's name next to their team's name (heroes without a team must also appear).

```
select h.name, t.name from hero h join team t on
h.team_id = t.id
```

3. The number of heroes per team.

```
select t.name as team_name, count(h.id) as number_of_heroes
from team t
left join hero h on
t.id = h.team_id
group by t.name, t.id
```

4. The 2nd page of heroes when a page has 2 rows, ordered by id .

SELECT * FROM hero ORDER BY id LIMIT 2 OFFSET 2;

? Question 1. For each of the four statements, which constraint blocked it (PRIMARY KEY , UNIQUE , NOT NULL , FOREIGN KEY) and why?

	Statement	Constraint blocked	Why
1	INSERT INTO team ... ('Avengers', 'Los Angeles')	UNIQUE (team_name_key	A team named Avengers already exists, and team.name is declared UNIQUE , so two rows cannot share the same name.
2	INSERT INTO hero (name, team_id) VALUES ('Ghost', 99);	FOREIGN KEY	hero.team_id references team(id) , and no team with id 99 exists. The database refuses a reference to a missing row.
3	INSERT INTO hero (age) VALUES (30)	NOT NULL	hero.name is NOT NULL and no value was given for it.
4	DELETE FROM team WHERE id = 1;	FOREIGN KEY	Heroes Tony and Natasha still reference team 1 through team_id . Deleting the team would leave them pointing at nothing.

? Question 2. The relationship team → hero is one-to-many. Why is the foreign key on hero and not on team ?

The foreign key is on **hero** because a hero belongs to **at most one** team, so a single **team_id** column in each hero row can store that relationship. Each cell holds exactly one value.

If the foreign key were on **team**, one team row would have to store many hero ids. That means either a list inside one column, which breaks the first normal form (atomic values), or duplicated team rows, one per hero. Both are wasteful and error-prone.

With the FK on the "many" side, adding a hero is a single **INSERT** into **hero** and the **team** table is never touched.

Rule of thumb: in a one-to-many relationship, the foreign key always goes on the "many" side.

? Question 3. Heroes can go on many missions and a mission has many heroes (many-to-many). Sketch the tables you need (names, columns, PK, FK). Hint: you need a link table

```
CREATE TABLE mission (  
    id SERIAL PRIMARY KEY,  
    title VARCHAR NOT NULL  
);
```

```
CREATE TABLE hero_mission (  
    hero_id INTEGER NOT NULL REFERENCES hero(id),  
    mission_id INTEGER NOT NULL REFERENCES mission(id),  
    PRIMARY KEY (hero_id, mission_id)  
); .
```

✓ Checkpoint 1: \dt shows Did not find any relations.

```
appdb-> \dt  
Did not find any relations.  
appdb->
```

Part 2 — Project Setup and DATABASE_URL (10')

✓ Checkpoint 2: echo \$DATABASE_URL (PowerShell: echo \$env:DATABASE_URL) prints the URL, and python -c "import sqlmodel, psycopg" runs without error.

```
(.venv) PS D:\database_web\hero_api> $env:DATABASE_URL = "postgresql+psycopg://app:secret@localhost:5432/appdb"  
(.venv) PS D:\database_web\hero_api> echo $env:DATABASE_URL  
postgresql+psycopg://app:secret@localhost:5432/appdb
```

```
(.venv) PS D:\database_web\hero_api> python -c "import sqlmodel, psycpg; print('OK')"
```

? Question 4. Why read the URL from an environment variable instead of writing it in database.py ? Give two reasons.

1. Security: The database URL may contain a username and password, so storing it in an environment variable prevents credentials from being exposed in the source code.

2. Flexibility: The same code can be used in different environments (development, testing, production) by changing the environment variable instead of modifying [database.py](#).

Part 3 — Define the Models (20')

3.1 Tables

? Question 5. Why is id typed int | None with default=None , when every row in the database has an id?

Because the database generates the ID when a new object is inserted. Before insertion, the Python object does not have an ID, so it is `None`. After insertion, the database assigns an integer ID.

? Question 6. Which attributes of Hero become columns, and which one does not? What is back_populates for?

The Hero columns are `id`, `name`, `age`, `team_id`, and `secret_name`. The `team` attribute is not a column; it is a relationship. `back_populates` connects the two sides of the relationship, allowing `hero.team` and `team.heroes` to access each other.

✓ Checkpoint 3: `python -c "from app.models import Hero, HeroPublic; print(Hero.__tablename__)"` prints hero .

```
(.venv) PS D:\database_web\hero_api> python -c "from app.models import Hero, HeroPublic; print(Hero.__tablename__)"
```

Part 4 – Engine, create_all, Session

? Question 7. Compare the CREATE TABLE hero printed by SQLAlchemy with the one you wrote by hand in Part 1. List the differences (types, NOT NULL , indexes, constraints).

Compared with my Part 1 table, the SQLAlchemy version adds the `secret_name` column because it is defined in the `Hero` model. The `name` column also has an index because `Field(index=True)` is used. The other columns have essentially the same types and constraints: `id` is a primary key, `name` is NOT NULL, `age` is nullable, and `team_id` is a foreign key referencing `team(id)`.

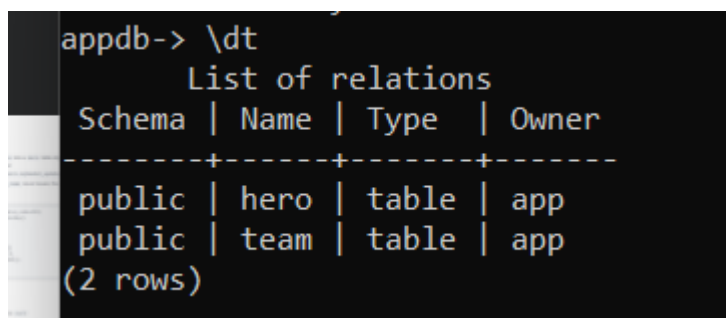
? Question 8. Stop and restart the server. Is CREATE TABLE printed again? Why? What does `create_all` do when a table already exists?

No. `CREATE TABLE` is not printed again because the tables already exist in the database. `create_all()` checks whether the tables exist and creates only tables that are missing. It does not drop or recreate existing tables.

? Question 9. `create_all` only knows about models that have been imported. Which line in `main.py` makes sure `Hero` and `Team` are registered?

The line is `from app import models`. It imports `models.py`, where `Hero` and `Team` are defined, so they are registered in `SQLModel.metadata` before `create_all(engine)` is called.

✓ Checkpoint 4: \dt lists hero and team . 📸 Screenshot it.



```
appdb-> \dt
          List of relations
Schema | Name | Type  | Owner
-----+-----+-----+-----
public | hero | table | app
public | team | table | app
(2 rows)
```

Part 5 — CRUD Endpoints for Heroes (30')

✓ Checkpoint 5: the heroes are still there after the restart (unlike Week 7). 📸 Screenshot it. All five endpoints return the status codes in the table above.

The screenshot displays a REST client interface at the top, showing a GET request to `http://127.0.0.1:8000/heroes` with a status of 200 OK. The response body is a JSON array of three hero objects: Peter (age 16, team_id null, id 1), Thor (age 500, team_id null, id 3), and bob (age 20, team_id null, id 4). Below this, a database interface shows a SQL query `SELECT * FROM hero;` with a result set containing three rows of hero data.

	name character varying	age integer	team_id integer	id [PK] integer	secret_name character varying
1	Peter	16	[null]	1	Spider-Man
2	Thor	500	[null]	3	God of thunder
3	bob	20	[null]	4	Sentry

? Question 10. Comment out `session.commit()` in `create_hero` and create a hero. What does the response look like, and is the row in the database (`SELECT * FROM hero;`)? Put the line back. What does `add()` do on its own, and why do we need `refresh()` ?

`add()` adds the object to the session but does not commit it to the database. Without `commit()`, the new row is not permanently stored. `refresh()` reloads the object from the database, including database-generated values such as the ID.

? Question 11. Which SQL statement does `echo=True` print for PATCH with body `{"age": 17}`? Does it update every column or only age? Why?

`echo=True` prints an UPDATE statement that changes only the `age` column, for example `UPDATE hero SET age=... WHERE hero.id=...`. It does not update every column because `model_dump(exclude_unset=True)` keeps only the fields actually provided by the client.

? Question 12. Look at the JSON returned by GET `/heroes/{id}`. Is `secret_name` there? Which line of code is responsible?

No, `secret_name` is not included in the JSON response. The `response_model=HeroPublic` line is responsible. `HeroPublic` does not define `secret_name`, so FastAPI excludes it from the response.

Part 6 — Teams, Filtering, Pagination and Errors (20')

6.1 Team endpoints with a 409 Conflict

```
INFO: 127.0.0.1:58564 - "POST /teams HTTP/1.1" 409 Conflict
```

6.2 Filter and sort in the database

? Question 13. Call GET `/heroes?min_age=18&team_id=1` and copy the SELECT printed by `echo=True`. Where do the values 18 and 1 appear? Why is this safe against SQL injection?

The values 18 and 1 appear as bound parameters (`age_1 = 18` and `team_id_1 = 1`), rather than being directly inserted into the SQL string. This is safe against SQL injection because SQLAlchemy uses parameterized queries, so user input is treated as data instead of executable SQL.

? Question 14. Why filter in the database instead of [h for h in session.exec(select(Hero)).all() if h.age >= 18] ?

Filtering in the database is more efficient because the database only returns rows that satisfy the condition. Filtering in Python first loads all rows into memory and transfers unnecessary data from the database. Database filtering also allows the DB optimizer and indexes to improve query performance.

6.3 Challenge: invalid foreign keys

✓ Checkpoint 6: duplicate team → 409 ; unknown team id → 404 ; the filters combine correctly.

```
2020-10-04 21:21:55,954 INFO sqlalchemy.engine.Engine ROLLBACK
INFO: 127.0.0.1:62371 - "GET /teams/99/heroes HTTP/1.1" 404 Not Found
```

Query

Query History

1

2

SELECT * FROM team;

Data Output

Messages

Notifications

≡+

📄

▼

📋

▼

🗑️

🗄️

⬇️

📈

SQL

	name character varying	headquarters character varying	id [PK] integer
1	Avengers	New york	1

Part 7 — Many-to-Many: Missions (15')

? Question 15. On restart, create_all did create mission and heromissionlink . In Part 4 it did nothing for hero . What is the rule?

The rule is that `create_all()` only creates tables that do not already exist. It does not recreate or modify existing tables. Therefore, `hero` already existed from Part 4, so nothing happened to it, while the new `mission` and `heromissionlink` tables did not exist, so they were created on restart.

✓ Checkpoint 7: after assigning hero 1 to mission 1 twice, heromissionlink contains exactly one row (1, 1) .

Query

Query History

1 `select * from heromissionlink`

Data Output

Messages

Notifications



SQL

	hero_id [PK] integer 	mission_id [PK] integer 
1	1	1

Part 8 — Seed Script (10')

? Question 16. You never set `team_id` in the seed script. Read the `echo=True` output: in which order were the INSERT s executed, and how did `hero.team_id` get its value?

The INSERTs were executed in dependency order: teams and missions were inserted first, then heroes, and finally the hero-mission link rows. `hero.team_id` got its value from the `team=avengers` relationship. After `avengers` was inserted, the database generated its ID, and SQLAlchemy automatically used that ID as `hero.team_id` when inserting the hero.

✓ Checkpoint 8: GET /teams/1/heroes returns the Avengers, GET /heroes/1/missions returns Battle of Sokovia.

GET http://127.0.0.1:8000/teams/1/heroes

Body

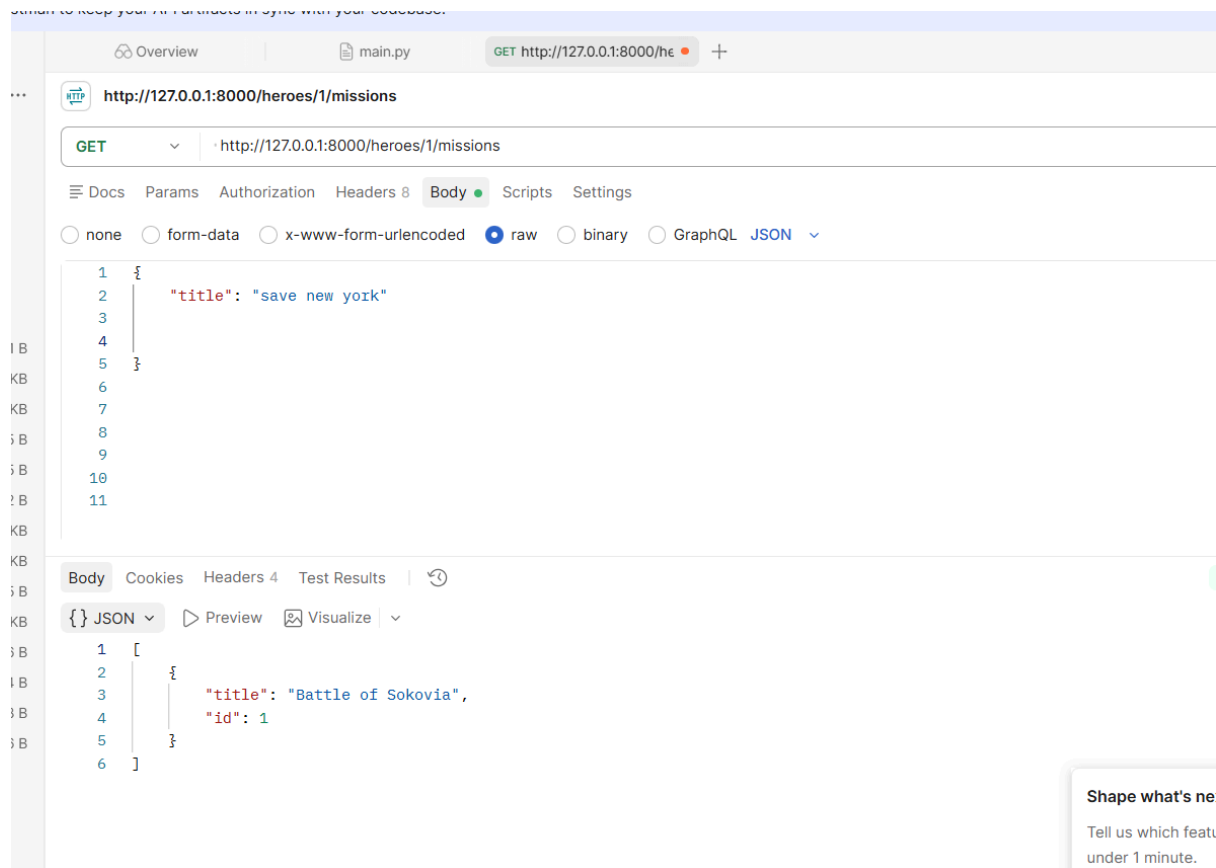
```
1 {
2   "title": "save new york"
3 }
4
5 }
6
7
8
9
10
11
```

200 OK • 124 ms • 219 B

JSON

```
2 {
3   "name": "Tony",
4   "age": 45,
5   "team_id": 1,
6   "id": 1
7 },
8 {
9   "name": "Natasha",
10  "age": 35,
11  "team_id": 1,
12  "id": 2
13 }
14 ]
```

Shape what's next
Tell us which features matter most to you. Take: under 1 minute.
Activate Windows
Go to Settings to activate Windows.
Not now Take the survey



Part 9 — Migrations with Alembic (20')

? Question 19. Where does Alembic store "which revision this database is at"? (Hint: \dt .) Why should the migrations/ folder be committed to Git?

Alembic stores the current database revision in the `alembic_version` table. The `migrations/` folder should be committed to Git because it contains the versioned migration history and instructions needed to consistently upgrade or downgrade the database schema across development, testing, and production environments.

? Question 20. Rename `secret_name` to `alias` in the model and run `revision --autogenerate` (do not apply it). What did Alembic generate? Why is that dangerous for existing data, and how would you fix the script? Afterwards, delete that revision file and undo the rename.

Alembic may generate a drop of `secret_name` and an add of `alias` instead of detecting a true column rename. This is dangerous because dropping `secret_name` can permanently delete existing data. The migration should be manually changed to use `op.alter_column("hero", "secret_name", new_column_name="alias")` in `upgrade()`, and the reverse rename in `downgrade()`. After inspecting the migration, the generated revision file should be deleted and the model changed back to `secret_name`.

✓ Checkpoint 9: alembic history shows your revision, alembic current shows (head) . 📷
Screenshot it.

```
(.venv) PS D:\database_web\hero_api> alembic history
<base> -> 7335354cdce5 (head), add hero.power
(.venv) PS D:\database_web\hero_api>
```

```
(.venv) PS D:\database_web\hero_api> alembic current
INFO [alembic.runtime.migration] Context impl PostgresqlImpl.
INFO [alembic.runtime.migration] Will assume transactional DDL.
(.venv) PS D:\database_web\hero_api>
```

Part 10 — Project Integration (group work, finish as homework)

Review Questions Answer without looking, then check.

1. What is a primary key for?

A primary key (PK) uniquely identifies each row in a table. It cannot contain duplicate or **NULL** values.

2. What is the main benefit of an ORM?

An ORM (Object-Relational Mapping) allows us to work with database data using programming language objects/classes instead of writing SQL for every operation.

For example:

```
hero = Hero(name="Tony", age=45)
session.add(hero)
session.commit()
```

3. What does `table=True` change about a `SQLModel` class?

`table=True` tells `SQLModel` that the class represents a database table.

```
class Hero(SQLModel, table=True):
```

Without `table=True`, the class is normally used as a data/schema model, not as a database table.

4. After `session.add(...)` , what actually saves the data to the database?

`session.add()` only adds the object to the session.

```
session.add(hero)
```

The transaction is actually saved permanently when we call:

```
session.commit()
```

So:

```
add() → put the object into the session  
commit() → commit the transaction to the database
```

5. What does `session.exec(select(Model)).all()` return?

It returns all matching records as model objects, usually as a list.

For example:

```
heroes = session.exec(select(Hero)).all()
```

returns a list of `Hero` objects.

6. What is Alembic for?

Alembic is used to manage database schema migrations.

It allows us to safely track and apply changes to the database schema, such as:

- adding a column
- removing a column
- renaming a column
- changing a table structure

For example:

```
alembic revision --autogenerate
```

creates a migration, and:

```
alembic upgrade head
```

applies it.

7. You add a field to a model and restart the app. Why is there no new column?

Because:

```
SQLModel.metadata.create_all(engine)
```

only creates missing tables. It does not modify tables that already exist.

For example, if the database already has:

```
hero
├── id
├── name
└── age
```

and you add:

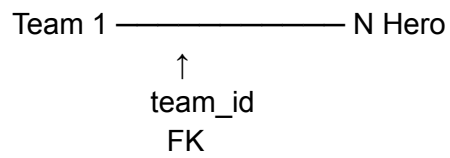
```
power: str | None = None
```

to the model, restarting the application does not automatically add **power** to the existing table.

You need a migration with Alembic.

8. In a one-to-many Team–Hero relationship, where does the foreign key go?

The foreign key goes on the many side.



For example:

```
class Hero(SQLModel, table=True):
    team_id: int | None = Field(
        default=None,
        foreign_key="team.id"
    )
```

So the rule to remember is:

In a one-to-many relationship, the foreign key is placed on the "many" side.